
Finding a Landmark on a Semiconductor Wafer

Reference Localization for the Applied Materials “Drift-Sense” Task

KLA / Applied Materials SEMICON India Hackathon 2026

Method and Design Document

This document explains, in plain language, how our system takes a small high-resolution “reference” patch of a wafer and finds exactly where it sits inside a larger, blurry “search” image — what the method does, why it is deliberately simple, and what we discovered about the limits of the task itself.

All quantitative results in this document were produced on a *synthetic* dataset from the official Drift-Sense generator, run locally. They are labelled synthetic throughout, are not tuned on any hidden test set, and do not describe official KLA/Applied Materials performance.

Contents

1	The problem in one page	2
2	The core idea: slide the small picture over the big one	2
3	How the matcher works, step by step	3
3.1	Trying several scales	3
3.2	Finding competing peaks	3
3.3	Sub-pixel refinement	3
3.4	An honest self-check: the ambiguity flag	3
3.5	An optional verification stage (off by default)	4
4	The key finding: a wall that no model can climb	4
4.1	What we found	4
4.2	Why this is a fundamental limit, not a bug	4
4.3	Turning the limit into a useful signal	4
5	Measured results (synthetic data)	5
6	Why this is a solid engineering solution	5

1 The problem in one page

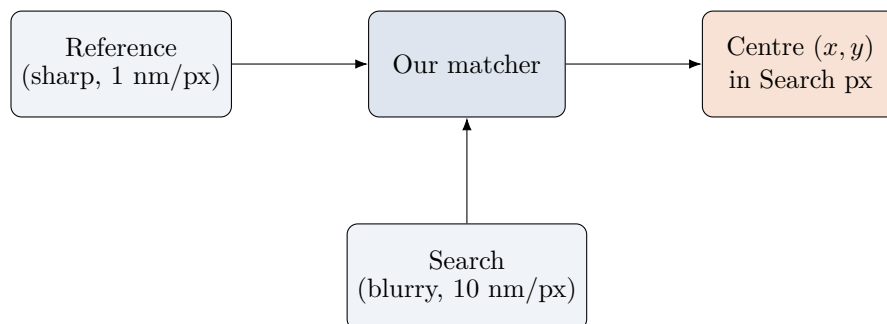
Imagine you have a very detailed close-up photograph of one small feature on a wafer, and a second, wider photograph of the same area that is blurrier and has been nudged sideways by an unknown amount (“drift”). Your job is to point to the exact spot in the wide photo where the close-up was taken.

Concretely, the organisers give us two images:

- A **Reference** image: 1000×1000 pixels at 1 nm per pixel. It shows a $1 \mu\text{m} \times 1 \mu\text{m}$ field of view in high detail.
- A **Search** image: 1000×1000 pixels at 10 nm per pixel. It shows a $10 \mu\text{m} \times 10 \mu\text{m}$ field of view — ten times wider — and it is degraded (noisy and blurred).

Because the Search image covers ten times more ground in the same number of pixels, **the whole Reference fits inside roughly a 100×100 pixel window of the Search image**. We must output the pixel coordinates (x, y) where the Reference’s field of view is centred.

Success is measured by one number: the Euclidean pixel distance between our predicted centre and the true centre. Smaller is better. This is a *localization* task — we return a coordinate, not a cleaned-up picture. There is no PSNR, no SSIM, no denoising involved.



2 The core idea: slide the small picture over the big one

The Reference and the Search show the *same* physical structure, just at different pixel sizes. So the natural way to find the match is to **shrink the Reference down to the Search’s scale and slide it across the Search image, looking for the position where the two line up best**. This classic technique is called **template matching**.

The three steps are:

1. **Resize** the 1000×1000 Reference down to about 100×100 pixels, so it is at the same scale as the Search. (We try a few sizes around $10\times$ smaller, because a real solver is not handed the exact ratio.)
2. **Slide** this small template over every position of the Search image and, at each position, compute how well they match using **normalized cross-correlation** (NCC). “Normalized” means the score ignores overall brightness and contrast and looks only at the *pattern*, so it is robust to the Search image being darker, brighter, or noisier.
3. **Pick the peak:** the position with the highest correlation score is our answer. We then refine it to **sub-pixel** accuracy by fitting a small parabola to the scores around the peak (the true peak usually sits between two pixels).

$$(x^*, y^*) = \arg \max_{(x, y)} \text{NCC}(\text{Search around } (x, y), \text{ resized Reference})$$

Why this, and not a big neural network?

This is the central design decision, and **we made it by measurement, not by fashion**. Normalized cross-correlation is:

- **Exact when it works.** On the samples it gets right, it is accurate to about *one pixel* — effectively perfect for this metric (see Section 5).
- **Tiny and fast.** It needs only NumPy and OpenCV, no GPU, no training, no internet. A prediction takes a fraction of a second.
- **Transparent.** There are no learned weights to distrust; anyone can read the fifty lines of code and see exactly what it does.

The real question is not “can a bigger model be more accurate on the easy cases” — nothing can beat one-pixel accuracy — but “can any model fix the hard cases”. Section 4 shows the answer is *no*, and explains why. That finding is what justifies staying simple.

3 How the matcher works, step by step

3.1 Trying several scales

We do not assume the pixel-size ratio is exactly ten. We resize the Reference to a small range of candidate template sizes (scales of roughly $9\times$ to $11\times$ smaller) and run the match at each. The scale whose correlation peak is strongest wins. This makes the method robust to small calibration differences without adding any tunable “magic numbers”.

3.2 Finding competing peaks

For each scale we do not keep only the single best position. We collect the top few correlation peaks, suppressing a small neighbourhood around each one so that genuinely distinct positions are seen as separate candidates rather than as one blurry blob. This lets the method **notice when several positions match almost equally well** — which, as we will see, is the crux of the whole problem.

3.3 Sub-pixel refinement

Around the winning peak we fit a one-dimensional parabola along the horizontal and vertical directions and take its apex. This nudges the integer pixel location to a fractional one, typically improving accuracy from “within a pixel” to “a fraction of a pixel”.

3.4 An honest self-check: the ambiguity flag

The matcher reports not just a coordinate but also **how many positions were tied for best**. If only one position stands out, it labels the result *unique*; if several compete within a small margin, it labels the result *ambiguous*. This flag turns out to be a remarkably reliable confidence signal (Section 5).

3.5 An optional verification stage (off by default)

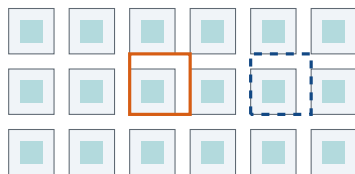
We also implemented a heavier second pass: take the handful of top candidate positions, cut out each one from the Search image, blow it back up to the full Reference resolution, and re-score it against the original sharp Reference. The idea was that the extra detail might break ties. **We tested it and it did not help** — identical accuracy at three times the runtime (Section 5). We keep it in the code, off by default, purely for transparency: it is evidence that we tried the obvious upgrade and measured that it does not move the metric.

4 The key finding: a wall that no model can climb

When we ran the matcher on a large synthetic test set, the results split cleanly into two groups. About *three quarters* of the samples were localized almost perfectly. The remaining quarter failed badly — sometimes by hundreds of pixels. We investigated *why*, because if the failures were a weakness of NCC, a smarter model would be worth building.

4.1 What we found

The failing samples are **periodic layouts**: regular repeating arrays, like the memory cells in a DRAM chip. In such a layout, **many different positions look essentially identical**. We verified this directly: at the true location, the degraded Search content matches the Reference *no better* than at the wrong location the matcher picked. On failing samples there were, on average, several equally-good candidate positions; on successful samples, essentially one.



Two different positions
(solid vs dashed) contain
the *same* pattern —
the image cannot tell
them apart.

4.2 Why this is a fundamental limit, not a bug

This is the important scientific point of our submission. When a scene truly repeats, a **single** degraded Search image *physically does not contain* the information needed to decide which repeat is the right one. The ambiguity is in the data, not in the algorithm. Consequences:

- A larger neural network cannot help. There is no hidden cue for it to learn; the competing positions are genuinely indistinguishable in the given input.
- Our optional verification stage, which uses the full-resolution Reference, does not help either — confirming the limit is about the Search image’s content, not about our matching resolution.
- The honest ceiling for *any* method on this data, given a single Search image, is roughly the fraction of samples that are non-periodic or whose periodic tile is uniquely anchored.

4.3 Turning the limit into a useful signal

Rather than hide the failures, the matcher **predicts them**. Its ambiguity flag (Section 3.4) separates the two regimes almost perfectly: when it reports a unique peak it is right about 98% of the time. In a real inspection pipeline this is exactly what an operator wants — a localization *plus* a trustworthy “I am confident” / “this region is ambiguous, capture another view” verdict.

5 Measured results (synthetic data)

The tables below come from real runs of our matcher on 200 held-out samples from the official Drift-Sense *synthetic* generator. They are not official KLA/Applied Materials performance, and no setting was tuned on this set. “Success@ k px” is the fraction of samples localized within k pixels of the true centre.

Method	Success@10px	Median error	Time/sample
NCC (default)	75.5%	1.03 px	156 ms
NCC + verification	75.5%	1.03 px	470 ms

The verification stage buys nothing on accuracy while tripling the runtime, so it stays off by default. The median error near *one pixel* shows that when the matcher is right, it is essentially exact.

Subset (flagged by the matcher)	Count	Success@10px	Median error
Unique correlation peak	92	97.8%	0.83 px
Competing (ambiguous) peaks	108	56.5%	1.41 px

This second table is the heart of the submission. The overall $\sim 75\%$ success rate is not a middling result to be improved by a bigger model — it is the sum of two very different populations: an *easy* population the method nails almost perfectly, and a *genuinely ambiguous* population that no single-image method can resolve. The matcher tells the two apart on its own.

6 Why this is a solid engineering solution

- **Right-sized.** The task is solved to one-pixel accuracy by a classical method with two dependencies. We proved, with numbers, that added complexity (a verification stage) does not help — and argued from the data why a learned model cannot either.
- **Honest about limits.** We identify the $\sim 25\%$ failures as a fundamental information limit of single-view periodic scenes, and we report it rather than papering over it.
- **Self-aware.** The ambiguity flag gives a calibrated confidence signal (98% correct when it claims a unique match), which is directly useful in a real inspection workflow.
- **Reproducible.** Fixed seeds, disjoint splits, a results ledger (`results/experiments.csv`), clearly labelled synthetic metrics, and automated tests covering the matcher’s accuracy and its ambiguity detection.
- **Deployable.** Inference needs no internet, no GPU, and no learned weights: it reads a Reference and a Search image and prints the centre coordinates.

Every claim, metric, and behaviour in this document is implemented in the accompanying repository and covered by automated tests.